

Agent: An agent's mind is a graph of beliefs

- Nodes: Beliefs - each beliefs has a value: measured by how worthwhile it is to reach.

Example: "survival" is highly valuable (+5); "danger" is "-2"

- Edges: Associations between beliefs - Each edge is directed (A->B != B->A) and has a weight (w). Weight is defined has how strongly one belief leads to another

the graph structure — which beliefs connect to which, and how strongly is the agent's personality

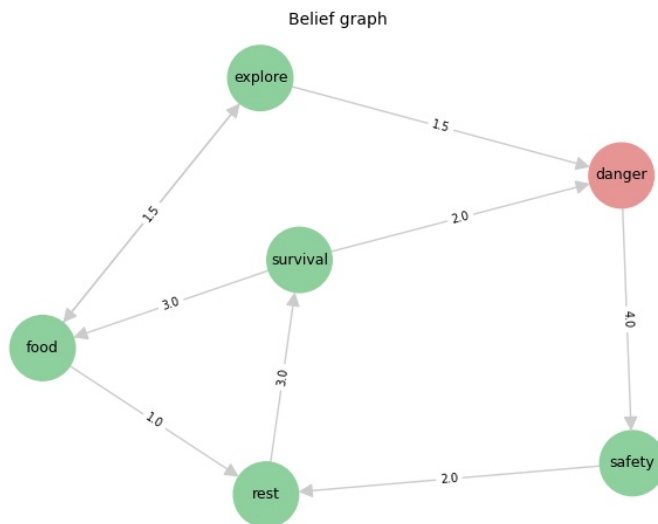
//graph.py

Using **networkx** library for graphs

Example used:

```
beliefs = {
    "survival": 5.0,
    "food": 3.0,
    "safety": 3.0,
    "rest": 2.0,
    "explore": 4.0,
    "danger": -2.0,
}
```

```
edges = [
    ("survival", "food", 3.0),
    ("survival", "danger", 2.0),
    ("food", "explore", 2.0),
    ("food", "rest", 1.0),
    ("danger", "safety", 4.0),
    ("safety", "rest", 2.0),
    ("rest", "survival", 3.0),
    ("explore", "danger", 1.5),
    ("explore", "food", 1.5),
]
```



//thought.py

G.successors(current_node) gives the beliefs you can reach from the current one by following outgoing arrows. From "survival" that's ["food", "danger"] (the two arrows leaving survival in your diagram).

The list comprehension **[G[current_node][n]["weight"] for n in neighbors]** collects the weight of each of those outgoing edges. For survival that's [3.0, 2.0] (food's edge and danger's edge). **G[a][b]** ["weight"] is how you read an edge's weight in networkx.

random.choices(neighbors, weights=weights, k=1) is Python's built-in weighted picker: give it options and matching weights, it returns one, biased toward the heavier weights. **k=1** = pick one; **[0]** unwraps the single-item list it returns.

Counter({'food': 605, 'danger': 395})

//thought_loop.py

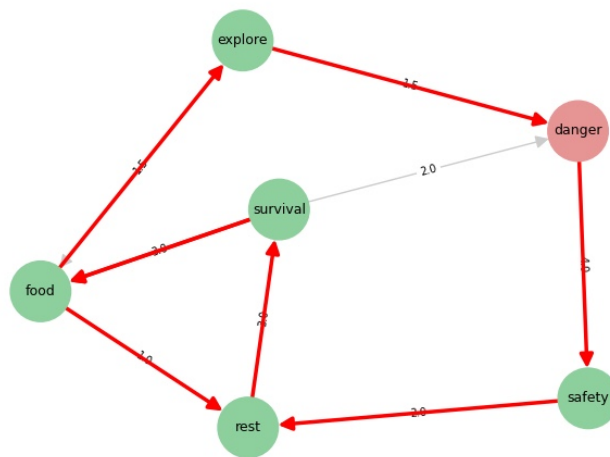
thought: described as many steps chained together until budget is reached.

max_steps: is the recurrence budget

trace: is the thought

Ex: survival -> food -> rest -> survival -> food -> explore -> danger -> safety -> rest

Belief graph: survival -> food -> rest -> survival -> food -> explore -> danger -> safety -> rest



//coverage.py

To introduce measurement that makes one thought more productive than another

Coverage — the fraction of all beliefs in the graph that a thought reaches at least once.

Result:

coverage=1.0 survival -> danger -> safety -> rest -> survival -> food -> explore -> danger -> safety -> rest -> survival

coverage=0.67 survival -> danger -> safety -> rest -> survival -> danger -> safety -> rest -> survival -> danger -> safety

coverage=1.0 survival -> food -> rest -> survival -> food -> explore -> danger -> safety -> rest -> survival -> food

coverage=1.0 survival -> danger -> safety -> rest -> survival -> danger -> safety -> rest -> survival -> food -> explore

Problem: Looping through same thought is useless and need to introduce better metric to improve measurement

payoff: Total value of the distinct beliefs a thought reaches.

Value is collected on FIRST visit only — revisiting a belief earns nothing, so a thought that loops without reaching anything new stops scoring. """

coverage=1.0 payoff=15.0

survival -> danger -> safety -> rest -> survival -> danger -> safety -> rest -> survival -> food -> explore

coverage=1.0 payoff=15.0

survival -> food -> explore -> danger -> safety -> rest -> survival -> food -> explore -> food -> explore

coverage=0.67 payoff=8.0

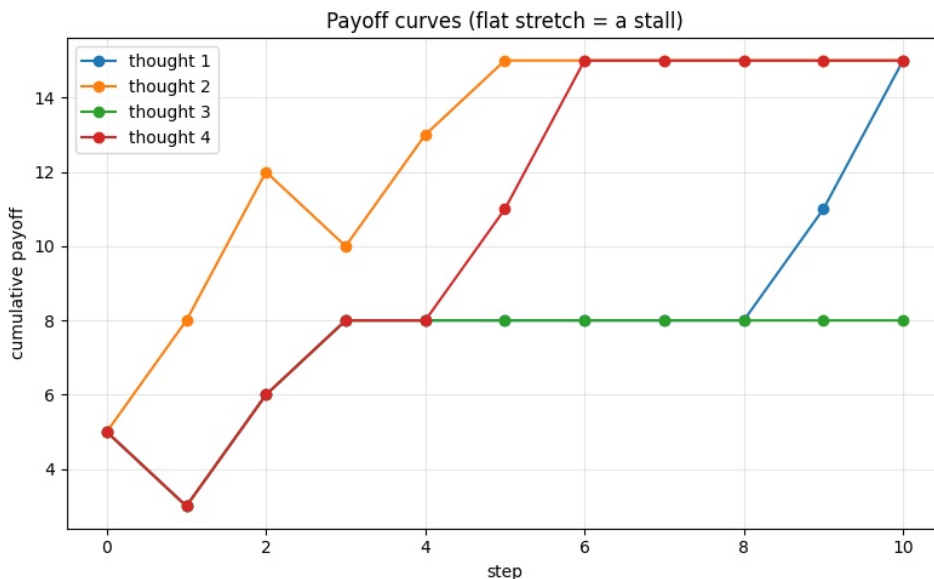
survival -> danger -> safety -> rest -> survival -> danger -> safety -> rest -> survival -> danger -> safety

coverage=1.0 payoff=15.0

survival -> danger -> safety -> rest -> survival -> food -> explore -> food -> explore -> food -> explore

thought_loop_tracked

To record payoff after every step to notice growth of thought and explore future payoff



Problem: Fix Stalling

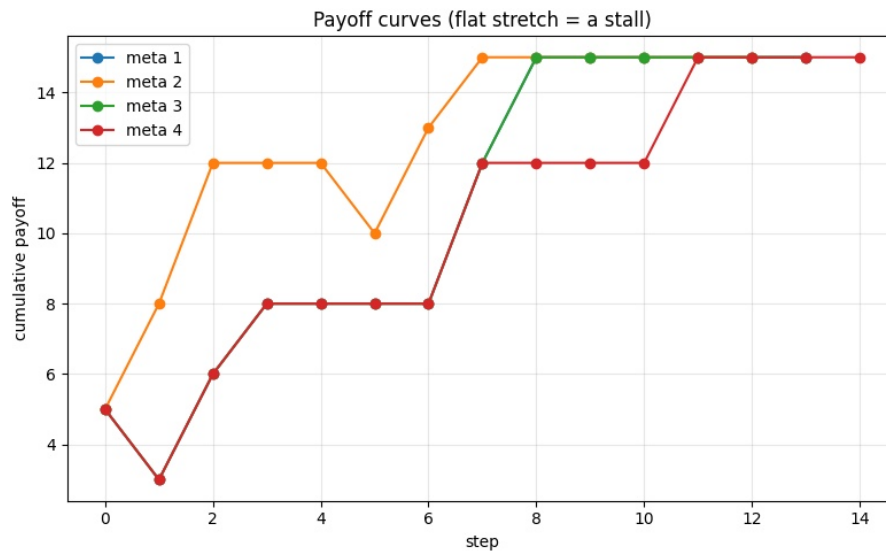
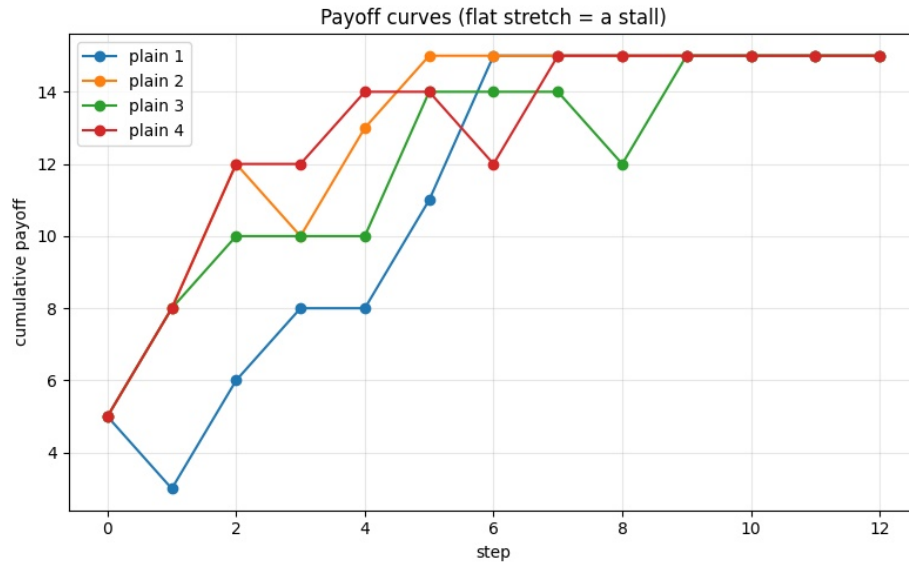
Solution: To introduce "meta-cognition" - create a monitor that watches payoff curve, when it has stalled for a certain amount of steps (patience is used to monitor the steps stalled) , the thought is intervened

Intervention:

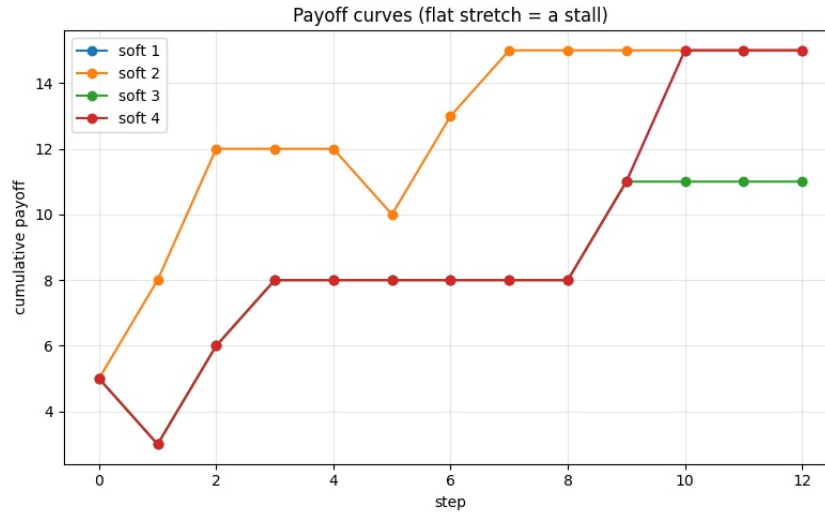
Hard jump- teleport the thought to the best unvisited belief.

Soft nudge — gently bias the dice toward unvisited beliefs, but let the walk still choose.

PATIENCE = 3 / HARD JUMP

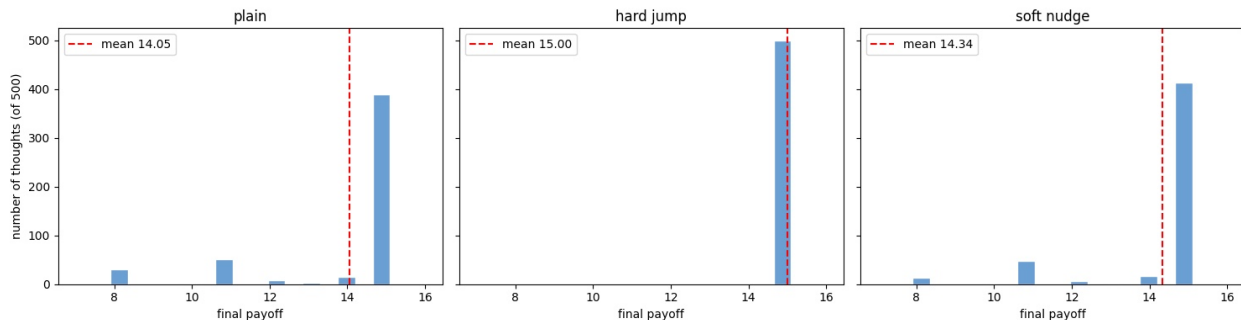


SOFT NUDGE



COMPARISON: PLAIN VS HARD VS SOFT

plain mean 14.05 stdev 2.02 min 8.0 max 15.0
 hard jump mean 15.00 stdev 0.00 min 15.0 max 15.0
 soft nudge mean 14.34 stdev 1.62 min 8.0 max 15.0



//Generators.py

cluster is just a group of beliefs wired into a closed loop.

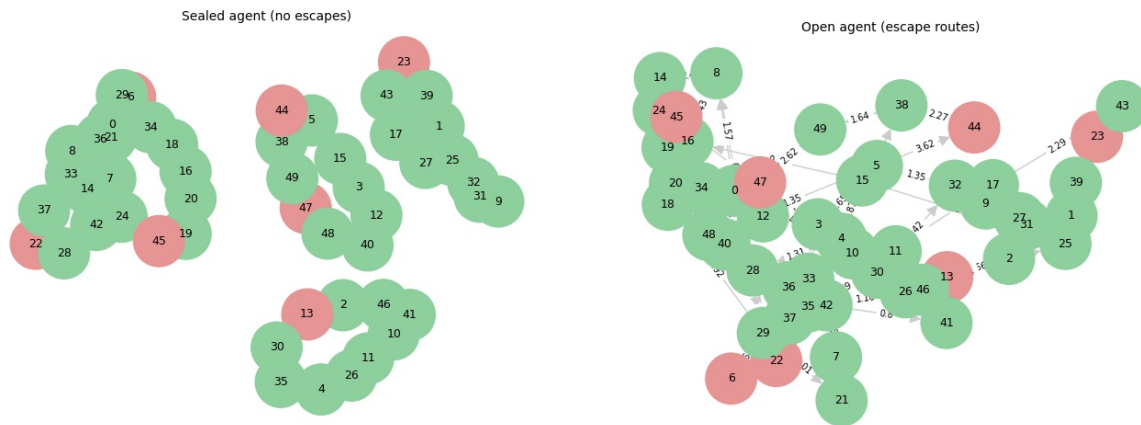
escape route is an extra connection that links one cluster to another

A **sealed agent** (no escapes): a thought can only ever reach the ~10 beliefs in its starting cluster. It collects those and then just loops, earning nothing more. Its payoff is capped low — it literally cannot reach most of its own beliefs.

An **open agent** (escape routes): a thought can travel across clusters, reaching many more beliefs, collecting far more value. Higher payoff.

sealed = build_agent(n_nodes=50, escape_density=0.0, seed=42) # trapped

open_ = build_agent(n_nodes=50, escape_density=0.6, seed=42) # escape routes



fitness.py

Create measurement to where "good" an agent is.
 fitness = average final payout over many thoughts from a random start node

//evolution.py

Reproduction: two parent agents make a child by combining their belief graph, inherits mix of both parents nodes and edges

Mutation: child wiring gets a small random changes, a few edge weights nudged , an edge may added or be removed

Selection: Evaluate agent's fitness and fitter agents are more likely to be chosen as parents

def crossover:

Node

$\text{set}(\text{parentA.nodes}) \mid \text{set}(\text{parentB.nodes})$ is the union of both parents' nodes — the $\mid$ operator on sets means "everything in either one." So the child gets every belief that appears in either parent. For each node, it collects that node's value from whichever parent(s) have it, and averages them ($\text{sum}(\text{vals}) / \text{len}(\text{vals})$). If both parents have node 5, the child's node 5 gets the average of their two values; if only one parent has it, the child just takes that one's value. So the child's beliefs are a merged, averaged set.

Edge

It walks through every edge that exists in either parent, and checks:

Both have it (inA and inB): the child definitely gets it, with the average of the two weights. The logic: if both parents independently have this connection, it's "stable"/important wiring, so it reliably passes on.

Only one has it (elif $\text{rng.random}() < 0.5$): a coin flip — 50% chance the child inherits it. So quirky wiring that only one parent had might pass on, might not. This is where variation gets shuffled: the child is a genuinely different mix each time.

def mutate:

Each edge has a 10% chance (`weight_rate=0.1`) of having its weight nudged up or down by a small random amount (`rng.uniform(-0.5, 0.5)`).

Add an edge (`add_edge_rate=0.05`, 5% chance): picks two random nodes and, if they're not already connected, adds a brand-new edge.

Remove an edge (`remove_edge_rate=0.02`, 2% chance): removes a random edge — but only if the source node has more than one outgoing edge (`H.out_degree(a) > 1`).

def next_generation:

Sorts agents by fitness (highest first) and copies the top elite (default 2) into the new generation unchanged.

Fills the rest of the population with children. The crucial line is `random.choices(agents, weights=fits, ...)` — it picks a parent with probability proportional to fitness. An agent with fitness 40 is roughly twice as likely to be picked as one with fitness 20. So fit agents reproduce more, passing their good wiring on more often. Each child is `mutate(crossover(...))` — blend two parents, then apply small mutations. This is natural selection in code.

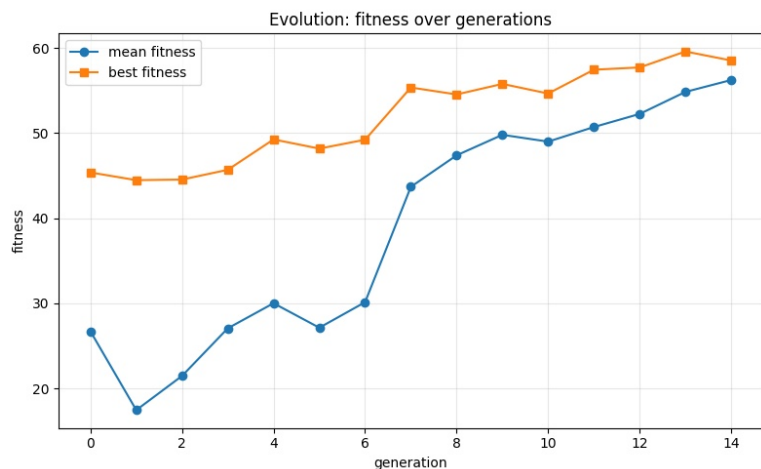
def evolve:

Each generation: score every agent (`mean_fitness` = the average payoff number from your fitness module), compute the population's mean and best fitness

simulation results:

gen | mean fitness | best fitness

0	26.87	46.95
1	20.09	46.29
2	24.68	46.48
3	18.62	47.79
4	30.55	47.46
5	43.57	47.95
6	44.22	47.08
7	42.85	47.67
8	44.72	46.58
9	45.84	48.77
10	45.69	49.00
11	46.26	49.69
12	46.26	47.87
13	45.54	47.89
14	44.98	47.40



//culture.py

CULTURE:

Agents are now put on a 2d grid, each agent in a cell. an agent interacts with its immediate neighbors (to introduce locality)

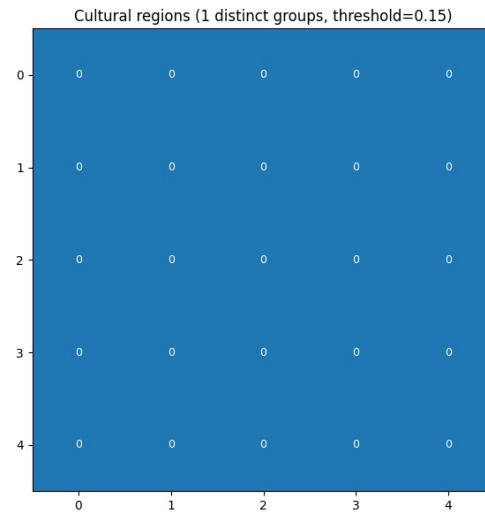
When two neighbors interact **copy-from-fitter** allows less fit neighbor to copy one belief edge from fitter neighbor (introducing conformity)

Problem: Do distinct local cultures form? (introducing clustering to measure similarity)

Simulation:

round | neighbor sim | random sim | clustering

0		0.030		0.022		+0.008
3		0.056		0.025		+0.031
6		0.080		0.033		+0.047
9		0.102		0.036		+0.066
12		0.122		0.047		+0.075
15		0.140		0.050		+0.090
18		0.157		0.048		+0.109
21		0.172		0.051		+0.121
24		0.187		0.060		+0.127
27		0.202		0.077		+0.125
29		0.211		0.061		+0.151

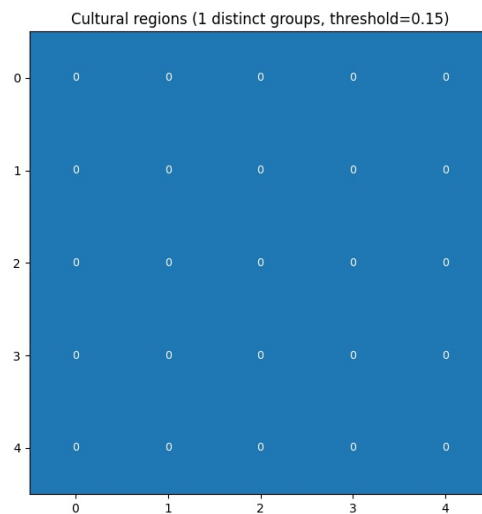


copy-from-fitter on a grid produces global convergence, not distinct subcultures.

ADDING HOMOPHILY: ONLY COPY FROM SIMILAR ENOUGH NEIGHBORS

**history, pop, coords = run_culture_with_snapshot(grid=5, rounds=30, seed=42,
homophily=0.15)**

n_groups = draw_culture_grid(pop, coords, grid=5, threshold=0.15)



homophily=0.15: neighbor_sim 0.215, clustering +0.153
homophily=0.35: neighbor_sim 0.206, clustering +0.141
homophily=0.55: neighbor_sim 0.212, clustering +0.153

NO distinct cultures have formed

